Functional Specification Document — Robô 2WD com ESP32

---

1. Document Control

Campo	Valor
Title	FSD — Firmware de Controle para Robô Diferencial 2WD com ESP32
Status	Draft / Pending Review
Target Version	1.0.0
Data	2026-11-04
Revisão	v1.1 — Open Questions resolvidas; imagens dos componentes analisadas

---

2. Executive Summary & Objectives

2.1 High-Level Description

Desenvolvimento do firmware embarcado para um robô de tração diferencial (2WD) baseado no microcontrolador **ESP32 WROOM32** (com placa de expansão Goouuu 38P/V4), capaz de executar uma trajetória de teste em formato de **quadrado** com controle de velocidade em malha fechada, utilizando encoders Hall de 12PPR como realimentação. O firmware será desenvolvido utilizando o **Arduino Framework** para ESP32.

2.2 Problem Statement

Sem realimentação de posição/velocidade, robôs de tração diferencial apresentam desvios de trajetória causados por diferenças de velocidade entre os motores (desbalanceamento mecânico e elétrico). O firmware deve corrigir essas diferenças em

tempo real via leitura dos encoders Hall, garantindo trajetórias precisas e reproduzíveis a velocidades abaixo do limite máximo dos motores.

2.3 Out of Scope (v1.0.0)

As funcionalidades abaixo estão **explicitamente fora do escopo desta versão** para evitar expansão não planejada:

- Controle remoto via Wi-Fi, Bluetooth ou UART externo.
- Mapeamento de ambiente (SLAM) ou desvio de obstáculos.
- Interface de usuário (display, botões físicos, app mobile).
- Controle PID auto-ajustável (auto-tuning).
- Odometria acumulada persistente em memória não-volátil.
- Integração com sensores externos (LiDAR, câmera, IMU).
- Uso do modo “mixed control” do ESC (somente modo “independent control” será utilizado).

---

3. Hardware de Referência

> *[!NOTE]*
> *As especificações abaixo foram extraídas diretamente das imagens dos componentes fornecidas.*

3.1 ESP32 com Placa de Expansão (Goouuu 38P/V4)

- **MCU**: ESP32 WROOM32
- **Placa de expansão**: Goouuu 38P/V4
- **Alimentação da placa**: DC 6.5V–16V (via conector DC) ou USB (Micro / Type-C)
- **Alimentação lógica disponível**: 3.3V e 5V nos headers
- **Conexão de programação/serial**: USB Micro ou Type-C

3.2 Motor DC com Encoder Hall (12PPR)

- **Tipo de encoder**: Hall Effect (magnético)
- **PPR (antes da redução)**: 12 pulsos por revolução do motor
- **Redução**: 1:90
- **PPR efetivo (na roda)**: 12 × 90 = **1080 pulsos/revolução da roda**
- **Pinout do conector**:

Pino	Descrição
`+`	Alimentação positiva do motor (da bateria/ESC)

`-`	Alimentação negativa do motor (da bateria/ESC)
`5V`	Alimentação da lógica do encoder (3V–12V)
`GND`	Terra da lógica do encoder
`A`	Sinal de pulso A (velocidade e posição)
`B`	Sinal de pulso B (sentido de rotação)

3.3 ESC para Motor Escovado — 2 Canais

- **Tensão suportada**: 2S–3S (6V–14V)
- **Modo de operação**: **Independent Control** (switch físico na placa do ESC deve estar nesta posição)
- **Canais**: CH1 e CH2, cada um controla um motor de forma independente
- **Inversão de direção**: Suportada via PWM (< 1500µs = ré, > 1500µs = frente)
- **Protocolo**: PWM padrão (1000µs–2000µs, 50Hz)

3.4 Chassi

- Metálico circular, 3 andares
- Roda de apoio castor na traseira (rolamento livre)
- 2 rodas motrizes laterais com motores DC

---

4. Parâmetros Físicos e Cálculos Derivados

Parâmetro	**Valor**	**Cálculo / Observação**
Diâmetro do chassi	140 mm	—
Distância entre rodas (track width)	120 mm	—
Diâmetro das rodas	70 mm	—
Circunferência das rodas	≈ 219.91 mm	`π × 70mm`
Redução do motor	1:90	—
PPR do encoder (antes da redução)	12	Encoder Hall magnético
PPR efetivo (na roda)	**1080**	`12 × 90`

Distância por pulso	**≈ 0.2036 mm/pulso**	`219.91mm / 1080`
Pulsos para 1 metro (linha reta)	**≈ 4911 pulsos**	`1000mm / 0.2036mm`
Pulsos por roda p/ girar 90°	**≈ 471 pulsos**	`(π × 120mm × 90/360) / 0.2036mm`
Pulsos por roda p/ girar 180°	**≈ 942 pulsos**	`(π × 120mm × 180/360) / 0.2036mm`
Tensão da bateria	7.4V (2S LiPo)	—

---

5. User Personas & User Journeys

5.1 Personas

Persona	Descrição
Desenvolvedor Embarcado	Responsável por programar, depurar e calibrar o firmware no ESP32 via USB/Serial usando Arduino IDE ou PlatformIO.
Operador de Teste	Liga o robô, posiciona na área de teste e observa a execução do quadrado.

5.2 User Journey — Ciclo de Teste Completo

1. **Setup**: O Desenvolvedor compila e grava o firmware via Arduino IDE / PlatformIO com a biblioteca ESP32 Arduino Core.
2. **Configuração do ESC**: O Operador posiciona o switch do ESC em **"independent control"** antes de energizar o sistema.
3. **Inicialização**: O Operador conecta a bateria 2S (7.4V) ao ESC. O firmware envia sinal de neutro (1500µs) a ambos os canais por 2 segundos para o protocolo de arm do ESC.
4. **Execução do Quadrado**: O robô executa automaticamente a trajetória: `frente` `giro 90°` `frente` `giro 90°` `frente` `giro 90°` `frente` `giro 90°` (4 lados + 4 curvas), retornando ao ponto inicial.
5. **Realimentação por Encoder**: A cada ciclo de controle (≤ 20ms), o firmware lê os pulsos dos encoders Hall, calcula a velocidade real de cada roda e corrige o PWM via controlador PID.

6. **Finalização**: Ao completar o quadrado, o firmware para ambos os motores e transmite métricas via Serial a 115200 baud.
7. **Análise**: O Desenvolvedor lê os logs via Serial Monitor e ajusta os ganhos PID conforme necessário.

---

6. Functional Requirements

ID	Módulo / Componente	Descrição do Requisito	Prioridade
FR-01	Inicialização de Hardware	O sistema DEVE configurar os GPIOs 18 e 19 como saídas PWM (canal LEDC do ESP32) para CH1 e CH2 do ESC, respectivamente, durante o `setup()`.	P0
FR-02	Inicialização de Hardware	O sistema DEVE configurar os GPIOs 23, 22, 33 e 32 como entradas digitais com `INPUT_PULLUP` e interrupção de borda (`CHANGE`) para leitura em quadratura dos Encoders 1 e 2.	P0
FR-03	Inicialização de Hardware	O firmware DEVE alimentar a lógica dos encoders Hall com 5V a partir da placa de expansão (header 5V disponível na Goouuu 38P/V4).	P0
FR-04	Controle de Motor	O sistema DEVE gerar sinais PWM com frequência de 50Hz (período 20ms) e largura de pulso entre 1000µs e 2000µs, via biblioteca `ESP32Servo` ou LEDC nativo do Arduino Core para ESP32.	P0
FR-05	Controle de Motor	O ESC DEVE operar em modo **independent control**. CH1 controla o Motor 1 e CH2 controla o Motor 2 de forma totalmente independente.	P0
FR-06	Controle de Motor	O sinal de neutro é **1500µs**. Valores acima de 1500µs movem o motor para frente; valores abaixo de 1500µs movem o motor para trás (inversão de direção suportada pelo ESC).	P0

FR-07	Controle de Motor	O sistema NÃO DEVE exceder ±35% do range de velocidade (equivalente a [1150µs, 1850µs]) durante a execução da trajetória de teste v1.0.0.	P0
FR-08	Leitura de Encoder	As ISRs de interrupção DEVEM incrementar ou decrementar um contador `volatile int32_t` para cada encoder usando lógica de quadratura (canais A e B) para determinar direção.	P0
FR-09	Leitura de Encoder	O sistema DEVE calcular a velocidade de cada roda em RPM a cada ciclo de controle: `RPM = (Δpulsos / PPR_efetivo) × (60.000ms / T_ms)`, com `PPR_efetivo = 1080`.	P0
FR-10	Controlador PID	O sistema DEVE implementar um controlador PID independente para cada motor (PID_Motor1 e PID_Motor2), com ganhos `Kp`, `Ki`, `Kd` definidos em `config.h`.	P1
FR-11	Controlador PID	O ciclo de controle PID DEVE ser executado a cada ≤ 20ms, acionado por timer de hardware via `hw_timer_t` do Arduino Core ESP32 ou `Ticker`.	P0
FR-12	Controlador PID	A saída do PID DEVE ser saturada no intervalo [1150µs, 1850µs] para evitar saturação do ESC.	P0
FR-13	Trajetória — Quadrado	A trajetória de teste DEVE executar **4 lados em linha reta** intercalados por **4 rotações de 90° no sentido horário**, retornando ao ponto de partida.	P0
FR-14	Trajetória — Primitivo Linha Reta	O primitivo `moverFrente(float distancia_mm)` DEVE acionar ambos os motores com velocidade alvo configurável e parar quando os encoders acumularem `distancia_mm / 0.2036 mm_por_pulso` pulsos.	P1

FR-15	Trajetória — Primitivo Rotação	O primitivo `girar(float angulo_graus, int sentido)` DEVE acionar os motores em direções opostas (Motor1 frente + Motor2 ré para girar à direita) e parar ao acumular `(π × track_width_mm × angulo/360) / 0.2036` pulsos em cada roda.	P1
FR-16	Trajetória — Parâmetros	O lado do quadrado e o ângulo de giro DEVEM ser constantes configuráveis em `config.h`. Valores padrão: `LADO_MM = 500` (50cm) e `ANGULO_GIRO = 90.0`.	P2
FR-17	Logging & Telemetria	Ao final de cada lado e cada giro da trajetória, o firmware DEVE imprimir via `Serial` (115200 baud) os pulsos acumulados de cada encoder e o RPM médio.	P1
FR-18	Logging & Telemetria	Ao finalizar o quadrado completo, o sistema DEVE transmitir o log de sumário no formato especificado na seção 7.5.	P1
FR-19	Segurança	O sistema DEVE parar ambos os motores (1500µs = neutro) caso a diferença de velocidade entre os dois motores supere 30% da velocidade alvo por mais de 500ms consecutivos, e DEVE emitir mensagem `[ERRO] Desbalanceamento detectado` via Serial.	P1
FR-20	Configurabilidade	Todos os parâmetros físicos, GPIOs, ganhos PID e parâmetros de trajetória DEVEM estar centralizados em `config.h`.	P2

---

7. Data Flow & Interface Specifications

7.1 Mapeamento de GPIOs

Sinal	GPIO	Direção	Protocolo / Notas
ESC CH1 (Motor 1)	GPIO18	Saída	PWM 50Hz, 1000–2000µs

ESC CH2 (Motor 2)	GPIO19	Saída	PWM 50Hz, 1000–2000µs
Encoder 1 — Canal A	GPIO23	Entrada (IRQ)	Hall, quadratura — velocidade/posição
Encoder 1 — Canal B	GPIO22	Entrada (IRQ)	Hall, quadratura — direção
Encoder 2 — Canal A	GPIO33	Entrada (IRQ)	Hall, quadratura — velocidade/posição
Encoder 2 — Canal B	GPIO32	Entrada (IRQ)	Hall, quadratura — direção
5V encoder (lógica)	Header 5V (expansão)	Saída	Alimenta pino 5V dos encoders Hall
GND encoder	GND (expansão)	Referência	GND compartilhado

7.2 Protocolo ESC — Mapeamento de Velocidade

Largura de Pulso	Comportamento
1000µs	Motor parado / arm mínimo
1500µs	**Neutro — motor parado em operação**
1150µs–1499µs	Marcha ré (velocidade crescente)
1501µs–1850µs	Marcha frente (velocidade crescente)
2000µs	Velocidade máxima frente (não usar em v1.0)

> *[!IMPORTANT]*
> *O switch físico do ESC DEVE estar na posição **"independent control"** para que CH1 e CH2 controlem os motores de forma independente. No modo "mixed control", o ESC interpreta CH1 como aceleração total e CH2 como direção (modo RC), o que é incompatível com o controle PID por roda.*

7.3 Fluxo de Dados — Ciclo de Controle PID

```
[Timer HW @ 50Hz (a cada 20ms)]
```

```
[Leitura Atômica dos Contadores de Encoder]
 (desabilitar IRQs    copiar volatile    reabilitar IRQs)
 Encoder1: GPIO23(A) + GPIO22(B)
 Encoder2: GPIO33(A) + GPIO32(B)


[Cálculo de RPM por Roda]
 Δpulsos = contador_atual - contador_anterior
 RPM = (Δpulsos / 1080) × (60.000 / 20)


[PID Motor 1]            [PID Motor 2]
 erro = RPM_alvo - RPM1    erro = RPM_alvo - RPM2
 saída = Kp*e + Ki*∫e + Kd*de/dt


[Saturação: clamp [1150µs, 1850µs]]


[writeMicroseconds(CH1) / writeMicroseconds(CH2)]


[Acumular odometria    verificar condição de parada]
```

7.4 Lógica de Quadratura dos Encoders Hall

```
Sentido Horário (frente):  A sobe antes de B    contador++
Sentido Anti-horário (ré): B sobe antes de A    contador--

ISR canal A:
  if (A == HIGH)    (B == LOW)  ? contador++ : contador--
  if (A == LOW)     (B == HIGH) ? contador++ : contador--
```

7.5 Interface Serial (Telemetria)

Log por segmento (após cada lado ou giro):

```
[SEG n] tipo=FRENTE dist_mm=500.0 enc1=4911 enc2=4908 rpm1_avg=35.2 rpm2_avg=3
```

```
[SEG n] tipo=GIRO   ang=90.0    enc1=471  enc2=470  rpm1_avg=20.1 rpm2_avg=20.0
```

Log de sumário final:

```
[ROBO_FIN] quadrado=OK enc1_total=XXXXX enc2_total=XXXXX tempo_total_ms=XXXXX
```

Log de erro:

```
[ERRO] Desbalanceamento detectado | rpm1=XX.X rpm2=XX.X diff=XX.X%
```

---

8. Arquitetura de Software

8.1 Estrutura de Pastas (Arduino Framework / PlatformIO)

```
firmware_robo2wd/
    platformio.ini        # Configuração PlatformIO (board: esp32dev, framework: arduir
    src/
        main.cpp          # setup(), loop(), sequência de trajetória
        config.h          # Todas as constantes configuráveis
        motor.h / motor.cpp # Abstração PWM via ESP32Servo ou LEDC
        encoder.h / encoder.cpp  # ISRs, contagem em quadratura, cálculo de RPM
        pid.h / pid.cpp     # Controlador PID genérico reutilizável
        trajectory.h / trajectory.cpp  # moverFrente(), girar(), executarQuadrado()
    lib/                  # Bibliotecas externas (ESP32Servo, etc.)
```

8.2 Conteúdo de `config.h` (Referência)

```
// === GPIOs ===
#define ESC_CH1_PIN    18
#define ESC_CH2_PIN    19
#define ENC1_A_PIN     23
#define ENC1_B_PIN     22
#define ENC2_A_PIN     33
#define ENC2_B_PIN     32

// === ESC ===
#define PWM_NEUTRO_US   1500
```

```
#define PWM_MIN_US      1150
#define PWM_MAX_US      1850
#define ESC_FREQ_HZ     50

// === Encoder & Mecânica ===
#define PPR_MOTOR       12
#define REDUCAO         90
#define PPR_EFETIVO     (PPR_MOTOR * REDUCAO)   // 1080
#define DIAM_RODA_MM    70.0
#define CIRC_RODA_MM    (PI * DIAM_RODA_MM)     // ~219.91mm
#define MM_POR_PULSO    (CIRC_RODA_MM / PPR_EFETIVO) // ~0.2036mm
#define TRACK_WIDTH_MM  120.0

// === Trajetória ===
#define LADO_QUADRADO_MM    500.0  // 50cm por lado
#define ANGULO_GIRO_GRAUS   90.0

// === PID ===
#define KP_MOTOR1   1.5
#define KI_MOTOR1   0.8
#define KD_MOTOR1   0.05
#define KP_MOTOR2   1.5
#define KI_MOTOR2   0.8
#define KD_MOTOR2   0.05

// === Controle ===
#define CICLO_CONTROLE_MS   20
#define RPM_ALVO_FRENTE     40.0
#define RPM_ALVO_GIRO       20.0
#define DESBAL_MAX_PERCENT  30.0
#define DESBAL_TIMEOUT_MS   500

// === Serial ===
#define BAUD_RATE   115200
```

---

9. Non-Functional Requirements

9.1 Performance

Requisito	Valor Alvo
Período do ciclo de controle PID	≤ 20 ms (50 Hz)
Latência da ISR de encoder	≤ 10 µs por borda detectada
Tempo de boot até início de movimento	≤ 3 segundos (incluindo 2s de arm do ESC)
Jitter do timer de controle	≤ 1 ms
Velocidade de operação — linha reta	≈ 40 RPM na roda (≤ 70% máx.)
Velocidade de operação — giro	≈ 20 RPM na roda

9.2 Confiabilidade

- Os contadores de encoder DEVEM ser do tipo `volatile int32_t` para garantir acesso correto entre ISR e loop principal.
- A leitura dos contadores no ciclo PID DEVE ser feita com seção crítica (`portDISABLE_INTERRUPTS` / `portENABLE_INTERRUPTS`).
- O firmware DEVE reiniciar os contadores de encoder via software entre segmentos de trajetória sem necessidade de reset de hardware.

9.3 Segurança Elétrica e de Software

- O firmware DEVE enviar `PWM_NEUTRO_US` (1500µs) a ambos os canais do ESC por no mínimo **2 segundos** no `setup()` , antes de qualquer movimento (protocolo de arm/calibração do ESC).
- Nenhum valor de PWM fora de [1150µs, 1850µs] DEVE ser gerado durante operação normal da v1.0.0.
- O firmware DEVE implementar parada de emergência por software caso a condição de desbalanceamento FR-19 seja detectada.

9.4 Manutenibilidade

- O código DEVE ser organizado em módulos separados conforme a estrutura da seção 8.1.
- Todos os parâmetros configuráveis DEVEM estar centralizados em `config.h` , sem literais numéricos (“magic numbers”) espalhados no código.
- O projeto DEVE incluir um `README.md` descrevendo: pinagem, como calibrar os ganhos PID e como alterar a trajetória de teste.

---

10. Acceptance Criteria

Inicialização

- **Dado** que o ESP32 é energizado com a bateria conectada e o switch do ESC em "independent control",
  Quando o firmware completa o `setup()`,
  Então ambos os canais ESC recebem 1500µs por ≥ 2 segundos antes de qualquer movimento.

Leitura de Encoder

- **Dado** que uma roda é girada manualmente uma volta completa,
  Quando o eixo completa 1 revolução da roda,
  Então o contador do encoder correspondente deve ter incrementado exatamente **1080 pulsos**.

Controle de Velocidade (PID)

- **Dado** que `RPM_ALVO_FRENTE = 40.0 RPM` está configurado,
  Quando o robô atinge regime permanente (após ≤ 3 segundos de movimento),
  Então a velocidade real medida por cada encoder deve estar dentro de **±10% de 40 RPM** (ou seja, entre 36 e 44 RPM).

Trajetória — Linha Reta

- **Dado** que `moverFrente(500.0)` é executado em superfície plana,
  Quando o robô completa o movimento,
  Então o desvio lateral medido fisicamente deve ser **≤ 3 cm** ao longo de 50cm de percurso.

Trajetória — Rotação

- **Dado** que `girar(90.0, HORARIO)` é executado,
  Quando o robô completa a rotação,
  Então o ângulo de giro real (medido com transferidor) deve estar dentro de **±8°** de 90°.

Trajetória — Quadrado Completo

- **Dado** que `executarQuadrado()` é chamado,
  Quando o robô completa os 4 lados e 4 giros,
  Então o robô deve retornar a uma posição final com desvio linear ≤ 10 cm e angular ≤ 15° em relação ao ponto de partida.

Parada de Segurança

- **Dado** que há um desbalanceamento simulado (carga em uma roda),
  Quando a diferença de velocidade entre os motores supera 30% por mais de 500ms,
  Então o firmware deve parar ambos os motores (1500µs) e emitir `[ERRO] Desbalanceamento detectado` via Serial.

Telemetria

- **Dado** que o quadrado completo é executado,
  Quando o robô para,
  Então o Serial Monitor deve exibir o log de sumário `[ROBO_FIN]` no formato da seção 7.5 dentro de ≤ 500ms após a parada.

---

11. Decisões de Design Registradas

#	Decisão	Justificativa
DD-01	Framework: **Arduino** (não ESP-IDF)	Menor curva de aprendizado; bibliotecas como `ESP32Servo` disponíveis; suficiente para os requisitos de v1.0.0.
DD-02	Protocolo ESC: **PWM padrão** (1000–2000µs @ 50Hz)	Compatível com o ESC de 2 canais identificado na foto3.
DD-03	Neutro em **1500µs** (não 1000µs)	ESC bidirecional usa 1500µs como ponto zero; 1000µs é o mínimo de arm, não o neutro operacional.
DD-04	ESC em modo **independent control**	Único modo compatível com controle PID por roda. Modo "mixed control" é para rádio controle convencional.
DD-05	Encoders alimentados com **5V** da placa de expansão	Encoder Hall opera em 3–12V; 5V disponível no header da Goouuu 38P/V4 garante margem de operação adequada.
DD-06	Trajetória fixa: **quadrado de 50cm de lado**	Suficiente para validar controle PID e odometria sem necessidade de área de teste grande. Parametrizável via `config.h`.

DD-07	PPR efetivo = **1080 pulsos/rev**	12 PPR (encoder Hall) × redução 1:90. Validado pela imagem do motor (foto4.png).